

PERÍODO 2 · LÓGICA, ALGORITMOS Y COMPUTACIÓN FÍSICA

8

Alertas

con lógica compuesta — validación con escenarios

Guía 18-8-TIC

LO QUE TE LLEVAS HOY

Sistema de alerta en MakeCode que combine 2-3 sensores con operadores AND/OR/NOT para producir al menos 3 niveles de alerta distintos (alarma máxima, alerta moderada, advertencia leve) + tabla de 8 escenarios de verificación que muestre cada combinación de entrada y la salida esperada vs. la salida real + bitácora corta de los ajustes hechos cuando alguna combinación no funcionaba como esperabas

ESTA GUÍA ES DE

Nombre completo

Grupo

Fecha

Autor: Dr. Álvaro Cárdenas Orozco

Metodología MILC · apertura ancestral · pensamiento computacional · triángulo Dussel–estoicismo–Floridi

Apertura: Alertas con lógica compuesta — validación con escenarios

Saber ancestral

En los puertos del Pacífico colombiano (Buenaventura, Tumaco, Bahía Solano) existía una figura silenciosa que sostenía la seguridad del comercio marino: **el centinela del puerto**. Sentado en la torre de vigilancia, con un catalejo viejo y un cuaderno cuadriculado, el centinela tenía la responsabilidad de **combinar varias observaciones simultáneas** para decidir qué nivel de alerta dar a la población costera. No era un trabajo de una sola variable: **exigía lógica compuesta de oficio profesional**. El centinela conocía sus reglas: “*Si veo barco grande Y bandera roja, alerta máxima*” (las dos cosas: AND). “*Si veo niebla O lluvia fuerte, alerta moderada*” (cualquiera de las dos: OR). “*Si NO hay viento favorable, alerta de retraso*” (la ausencia de algo: NOT). Esas combinaciones no eran arbitrarias: la práctica del oficio había decantado cuáles condiciones, juntas, justificaban cuál nivel de alerta. La sabiduría del centinela era doble: (1) **no dar alerta máxima por una sola señal sospechosa** (eso producía falsas alarmas y los marineros dejaban de creer). (2) **no esperar muchas señales para dar alerta moderada** (eso producía respuestas demasiado tardías). **La lógica compuesta vivía en la observación combinada**, mucho antes de que se llamara así en computación.

Saber contemporáneo

Un **sistema de alerta con lógica compuesta** combina las habilidades de las sesiones 1-7: leer sensores (S4-S6), aplicar umbrales calibrados (S6), combinar con AND/OR/NOT (S1-S2), depurar cuando falla (S7). La regla profesional: **los sistemas reales de alerta jamás dependen de una sola variable**. Una alarma de incendio combina sensor de humo Y sensor de temperatura alta (AND, para evitar falsos positivos por cocina); una alerta de robo combina detector de movimiento Y horario nocturno O ventana abierta (combinación de AND y OR); un sistema de riego inteligente activa el agua si humedad baja AND NOT está lloviendo AND horario permitido (combinación de las 3 lógicas). Para verificar que el sistema funciona, se construye una **tabla de escenarios**: cada fila representa una combinación de entradas, y se compara la salida esperada con la salida real. Con 3 sensores binarios, hay $2^3 = 8$ escenarios. Con 4, hay 16. La regla profesional: **ningún sistema de alerta se entrega sin haber sido probado en todos sus escenarios posibles**.

Pregunta-puente

*¿Qué sabía el centinela del puerto al combinar varias observaciones para decidir el nivel de alerta, que el programador novato olvida cuando hace un sistema de alarma con un solo sensor?
¿Y por qué probar 8 escenarios es la diferencia entre un sistema profesional y uno escolar?*

Saber y saber hacer

Al terminar podrás: (1) **identificar** qué combinación de 2-3 sensores produce alertas significativas para un problema real; (2) **aplicar** bloques de MakeCode con AND, OR, NOT para combinar lecturas de sensores con umbrales; (3) **analizar** los 2ⁿ escenarios posibles construyendo tabla de verificación; (4) **evaluar** el sistema corrigiendo aquellos escenarios donde la salida real no coincide con la esperada.

Autor	Dr. Álvaro Cárdenas Orozco
DBA	Pensamiento computacional aplicado a sistemas de control con sensores y actuadores (MEN, T&I)
Referentes	Pensamiento computacional · MakeCode / micro:bit · Logos como razón universal
Producto final	Sistema de alerta en MakeCode que combine 2-3 sensores con operadores AND/OR/NOT para producir al menos 3 niveles de alerta distintos (alarma máxima, alerta moderada, advertencia leve) + tabla de 8 escenarios de verificación que muestre cada combinación de entrada y la salida esperada vs. la salida real + bitácora corta de los ajustes hechos cuando alguna combinación no funcionaba como esperabas

→ Antes de empezar, mira el viaje completo. En las sesiones 1-7 aprendiste todas las piezas por separado. Hoy las integras en un sistema completo. La sesión 9 (proyecto) y la 10 (sustentación) tomarán esta integración como base. Es la cima técnica del periodo 2.

Ruta MILC: así vamos a aprender

1

Escucha
Observo, escucho
y pregunto.

2

Entender
Sistematizo
y ordeno.

3

Hacer
Construyo y aplico.

4

Cierre
Evalúo y reflexiono.

→ Antes de teorizar, vas a hacer un ejercicio del centinela: piensa en un problema real del colegio que necesite una alerta combinada (cerrar el aula al final del día, alertar cuando alguien entra a un lugar restringido, avisar si el aula está oscura Y hay alguien adentro) y anota qué sensores combinarías y con qué operadores.

Estación 1 de 3 · Escucha

Escena para escuchar

Actividad 1 · IDENTIFICA — Ejercicio del centinela (15 min · individual). Piensa en un problema real del colegio que necesite alerta combinada. Ejemplos: (a) **Cerrar el aula al final del día**: alerta si *hay personas adentro Y la hora es mayor a las 18:00*. (b) **Aula oscura ocupada**: alerta si *luz baja Y movimiento detectado* (alguien quedó sin luz). (c) **Permiso de entrada al laboratorio**: solo si *tarjeta válida AND NO es horario de clase Y NOT está cerrado por mantenimiento*. Elige uno y anota: ¿qué sensores usarías (luz, movimiento, botón)?, ¿qué operadores (AND, OR, NOT)?, ¿qué 3 niveles de alerta tendría (máxima, moderada, leve)?.

MARCA CUANDO LO HAYAS HECHO

- ☐ Elegí un problema real del colegio.
- ☐ Identifiqué qué sensores usaría.
- ☐ Definí 3 niveles de alerta.

Lo que va al cuaderno (Actividad 1 · IDENTIFICA). Título: «Actividad 1 — Ejercicio del centinela». **Formato**: ficha con problema + sensores + operadores + 3 niveles de alerta. **Sabes que terminaste cuando** tienes claro qué combinación produce cada nivel.

→ Ya tienes la idea. Ahora vas a entender la **anatomía del sistema de alerta profesional**: 3 niveles de alerta, combinación con AND/OR/NOT, tabla de escenarios de verificación, los 5 errores típicos del sistema novato (falsa alarma, alarma silenciosa, escenarios no probados, operadores confundidos, salida única para todo).

Estación 2 de 3 · Entender

Lo que vas a entender

Tu ficha del centinela define el problema. Ahora necesitas la **anatomía técnica**: cómo se programan los operadores compuestos en MakeCode, cómo se construye una tabla de escenarios, los errores típicos del sistema novato.

- 1 Los **bloques de lógica compuesta** en MakeCode (categoría Lógica) son: (1) si A y B entonces: bloque AND. Verdadero solo si ambas son verdaderas. (2) si A o B entonces: bloque OR. Verdadero si al menos una. (3) no A: bloque NOT. Invierte el valor. Se pueden anidar: si (A y B) o (no C) entonces. Los paréntesis se forman naturalmente con la estructura visual de los bloques.

2 La **tabla de escenarios** para 3 sensores binarios tiene $2^3 = 8$ filas con todas las combinaciones de Verdadero (V) y Falso (F). Cada fila tiene: (a) los valores de los 3 sensores. (b) la salida esperada según tu lógica. (c) la salida real al probar en el simulador. (d) ¿coinciden o no?. Si una fila no coincide, hay un bug que se depura con el método de la sesión 7.

3 Lo esencial cabe en una pregunta: *¿hay alguna combinación que mi sistema no haya considerado?*. Los sistemas de alerta novatos cubren los casos obvios (“si pasa esto, alerta”) y olvidan los casos extremos (“¿y si pasa solo una de las dos?”, “¿y si no pasa ninguna?”). La phronesis del oficio consiste en **construir la tabla completa antes de declarar el sistema funcional**.

4 Algoritmo: (1) definir entradas (2-3 sensores con sus umbrales). (2) definir salidas (3 niveles de alerta). (3) construir tabla de 2^n escenarios con salida esperada. (4) programar en MakeCode con if-else compuestos. (5) probar cada escenario en simulador y llenar tabla con salida real. (6) ajustar lo que no coincida (usando depuración de sesión 7). (7) documentar ajustes en bitácora.

Anatomía del sistema de alerta

Sensores: 2-3 con umbrales calibrados.

Operadores: AND, OR, NOT.

Niveles: 3 alertas distintas.

Tabla: 2^n escenarios con esperado vs real.

Verificación: cada fila probada.

Errores comunes y su corrección

(1) **Falsa alarma:** el sistema alerta cuando no debe (operador demasiado permisivo, OR donde debía AND). (2) **Alarma silenciosa:** el sistema no alerta cuando debería (operador demasiado restrictivo, AND donde debía OR). (3) **Escenarios no probados:** declarar el sistema funcional sin verificar todas las combinaciones. (4) **Operadores confundidos:** usar AND donde se quería OR o viceversa. (5) **Salida única:** 3 condiciones distintas pero solo 1 mensaje de alerta; el usuario no sabe qué pasó.

Lo que va al cuaderno (Actividad 2 · APLICA). Título: «Actividad 2 — Anatomía del sistema de alerta».

Formato: (a) bloques de lógica compuesta con ejemplo; (b) plantilla de tabla de escenarios para 3 sensores; (c) los 5 errores con marca. **Sabes que terminaste cuando** podrías construir una tabla de 8 escenarios para tu problema.

→ Tienes el método. Toca aplicarlo: programas un sistema con 2-3 sensores que produzca 3 niveles de alerta combinando AND/OR/NOT, lo verificas con tabla de 8 escenarios, ajustas lo que falle, documentas la bitácora.

Estación 3 de 3 · Hacer

Cómo vas a construir tu producto

Actividad 3 · ANALIZA + EVALÚA — Sistema completo + 8 escenarios verificados (40 min · individual o parejas). Hora de aplicar. Programar el sistema de alerta de tu elección con 2-3 sensores, construir tabla de 8 escenarios y verificar cada uno en el simulador.

- 1 Tu sistema se construye en 5 pasos: (1) definir entradas (2-3 sensores con umbrales calibrados de la sesión 6). (2) escribir tabla de escenarios con salida esperada (antes de programar). (3) programar en MakeCode con if-else compuestos. (4) probar cada escenario en simulador y llenar salida real. (5) ajustar lo que no coincida.
- 2 Sigue el patrón **“tabla antes que código”**: la tabla de escenarios se construye antes de programar. Eso obliga a pensar la lógica antes de implementarla. Si construyes la tabla después, tiendes a confirmar lo que ya programaste, no a verificar la lógica.
- 3 Para sistemas con 3 sensores se obtienen 8 escenarios. La regla: cada fila debe tener una salida definida (alerta máxima, moderada, leve, o ninguna). Si alguna fila quedó sin salida en tu tabla original, tu sistema tiene grietas lógicas que hay que cerrar.
- 4 Algoritmo: (1) definir entradas. (2) construir tabla con salida esperada. (3) programar. (4) probar las 8 filas. (5) anotar salida real. (6) si alguna no coincide, depurar (sesión 7). (7) registrar ajustes en bitácora.

Checklist antes de entregar

- ☐ 2-3 sensores definidos con umbrales calibrados.
- ☐ Lógica con AND, OR, NOT combinados.
- ☐ 3 niveles de alerta distintos.
- ☐ Tabla de 8 escenarios construida ANTES de programar.
- ☐ Cada escenario probado en simulador.
- ☐ Salida real coincide con esperada (o se documenta lo que falta).
- ☐ Bitácora de ajustes hecha.

Plantilla de trabajo

Problema. *Qué alerta resuelve.*

Sensores. *2-3 con umbrales.*

Operadores. *AND, OR, NOT combinados.*

Niveles. *3 alertas distintas.*

Tabla. *8 filas con esperado vs real.*

Bitácora. *Ajustes hechos.*

Lo que va al cuaderno (Actividad 3 · ANALIZA + EVALÚA). Título: «Actividad 3 — Sistema de alerta».

Formato: (a) descripción del problema + sensores + operadores; (b) tabla de 8 escenarios con esperado y real; (c) bitácora de ajustes; (d) referencia al código MakeCode. **Sabes que terminaste cuando** las 8 filas tienen salida coincidente, o se declaran las que aún no.

→ *Lo que sigue resume qué entregas hoy: sistema de alerta + tabla de 8 escenarios + bitácora de ajustes. El criterio principal: que el sistema funcione correctamente en todos los escenarios de la tabla, o que la bitácora explique honestamente cuáles aún fallan y por qué.*

Producto de la sesión

Sistema de alerta + tabla de 8 escenarios + bitácora de ajustes.

Criterios de calidad

(1) Sistema con 2-3 sensores combinados. (2) Lógica con AND, OR, NOT. (3) 3 niveles de alerta distintos. (4) Tabla de 8 escenarios construida antes. (5) Cada escenario verificado en simulador. (6) Bitácora de ajustes documentada.

→ *Antes de cerrar, mira el sistema desde las cinco dimensiones humanas. Probar todos los escenarios es respeto por quienes confiarán en la alarma; declarar funcional sin probar es negligencia con personas reales.*

Evaluación liberadora · cinco dimensiones**Sentido de la evaluación**

Evaluar no es solo revisar una nota. Es reconocer qué aprendí, qué cambió en mí, qué emociones manejé, cómo me relaciono con la ciudad y la comunidad, y qué le dejaré a quien viene detrás.

Mi reflexión liberadora en cinco dimensiones. Completa cada frase iniciada:

Dimensión	Mi respuesta (frame starter)	Algo que descubrí
1. Personal	Lo que más cambió en mí fue _____	_____
2. Emocional	La emoción más fuerte fue _____ y la regulé _____	_____
3. Ciudadana	En democracia esto importa porque _____	_____
4. Local (Cartago)	En mi barrio sería útil para _____	_____
5. Intergeneracional	A mi abuela/abuelo le diría _____	_____

Para la próxima sustentación. Vuelve a esta tabla en seis meses. Verás que las respuestas cambian — no porque lo aprendido cambie, sino porque tú cambias. La evaluación liberadora es una conversación contigo a través del tiempo.

→ Y para terminar, tres voces sobre los sistemas que protegen. Dussel sobre las alarmas que cuidan a las personas vulnerables, Marco Aurelio sobre la disciplina de probar todos los casos, Floridi sobre los sistemas de alerta como infraestructura ética digital.

Triángulo de pensamiento · cierre filosófico

Dussel · lente del nosotros

Una alarma que protege a quien depende de ella es liberadora; una que falla en silencio es traición técnica.

Los sistemas de alerta del mundo real protegen vidas: alarmas de incendio, sistemas de evacuación, detectores de fugas. La filosofía de la liberación pide construir esos sistemas con responsabilidad por las personas que dependerán de ellos. Tu sistema escolar de hoy te entrena en una práctica que mañana puede aplicarse a contextos con vidas reales en juego. La phronesis del oficio consiste en **construir alarmas como si la vida del usuario dependiera de ellas**, porque a veces literalmente lo hace.

Si mi sistema de alerta protegiera a alguien real, ¿confiaría en él tal como está, o falta verificación?

Estoicismo · Marco Aurelio · cuidado interior

Probar todos los escenarios es disciplina; declarar funcional con pruebas parciales es soberbia. Es tentador probar 2-3 escenarios obvios y declarar el sistema listo. La sabiduría estoica recomienda lo opuesto: *prueba los 8 escenarios completos*. Los bugs más graves suelen esconderse en escenarios extremos (todas las entradas falsas, una sola entrada verdadera). La disciplina de probar lo completo es virtud profesional.

¿Estoy probando los 8 escenarios completos o me detuve en los obvios?

Floridi · lente de la infoesfera

Los sistemas de alerta confiables son la infraestructura ética del oficio digital en la era de los riesgos automatizados.

En la era de la información, los sistemas de alerta automatizados sostienen muchas funciones críticas (transporte, salud, finanzas). Que esos sistemas sean confiables es responsabilidad ética del oficio digital. Tu hoja de hoy te entrena en esa práctica: construir sistemas de alerta verificados, documentados, transparentes. Esa disciplina escala desde el micro:bit escolar hasta sistemas industriales.

¿Mi sistema podría ser revisado por un auditor con la tabla de escenarios sin necesidad de mi explicación?

Nota para el docente. Las tres formulaciones del triángulo son la síntesis del autor de la guía, no citas textuales de los pensadores. Si vas a citarlos en clase, remite a la obra original.

Mi autoevaluación · marca una casilla por fila

Criterio	Lo logré	En proceso	Necesito apoyo
Comprensión del tema	☐ Explico las ideas con mis palabras.	☐ Reconozco las ideas, falta precisión.	☐ Necesito repasar lo básico.
Pensamiento computacional	☐ Usé los pilares al armar mi producto.	☐ Usé algunos pilares.	☐ Necesito releer la estación 2.
Aplicación y producto	☐ Mi producto cumple los criterios.	☐ Está armado, con ajustes pendientes.	☐ Aún está en construcción.
Reflexión 5 dimensiones	☐ Respondí cada una con honestidad.	☐ Respondí algunas con detalle.	☐ Mis respuestas son muy generales.
Triángulo de pensamiento	☐ Conecté las 3 voces con mi trabajo.	☐ Conecté al menos una.	☐ No las relaciono con mi proceso.

Hoy entendí que:

Mi compromiso para la próxima sesión es:

Cita de despedida · Marco Aurelio

“El obstáculo en el camino se vuelve el camino. Lo que estorba al camino se vuelve camino.”

Cada error de hoy, bien observado, se convierte en pieza del camino que pisarás mañana.

Nombre completo: _____

Fecha: _____ **Curso/grupo:** _____

Mi firma: _____

Para la próxima sesión. Llega con tu sistema de alerta y la tabla de 8 escenarios. En la sesión 9 vas a construir un **proyecto de monitoreo** usando el micro:bit como vigía nocturno de algún proceso real del colegio durante 3 días. La lógica compuesta de hoy es el motor; el proyecto de mañana le da contexto y propósito.